

Ridge vs. Lasso: Where the Circle and Diamond Come From

A detour into the geometry of regularized regression

Marcus Birkenkrahe

CS 540 – Summer 2026

Objective

This is a detour, independent of DataCamp and the textbook, that answers a question that comes up naturally once you've seen the "circle vs. diamond" picture used to explain why Lasso can zero out coefficients and Ridge can't: *where does that picture actually come from?* It's not an analogy — the ellipses and the constraint region are both direct consequences of the optimization problem `Ridge()` $\neq$ `Lasso()` are solving. This note derives both pieces.

The problem this explains

A model scores 0.95 R^2 on the training set and 0.60 on the test set. This gap is **overfitting**: the model has fit noise specific to the training data that doesn't generalize. The standard fix is **regularization** (Ridge or Lasso), which penalizes large coefficients to reduce model complexity.

Both add a penalty to the ordinary least squares (OLS) loss:

- Ridge (L2 penalty):

$$\min_{\beta} \text{RSS}(\beta) + \alpha \sum_j \beta_j^2$$

- Lasso (L1 penalty):

$$\min_{\beta} \text{RSS}(\beta) + \alpha \sum_j |\beta_j|$$

Both shrink coefficients toward zero as α grows. But only Lasso can drive a coefficient to *exactly* zero, which makes it useful for feature selection — Ridge shrinks everything a little but keeps every feature in the model. The usual textbook explanation for this difference is a picture: an "allowed region" (circle for Ridge, diamond for Lasso) that the loss function's contours run into. This note works out where that picture comes from.

Where the ellipses come from

RSS as a function of the coefficient vector β is a quadratic form:

$$\text{RSS}(\beta) = (y - X\beta)^T(y - X\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X \beta$$

Complete the square around the OLS solution $\hat{\beta} = (X^T X)^{-1} X^T y$, and this simplifies to:

$$\text{RSS}(\beta) = \text{RSS}(\hat{\beta}) + (\beta - \hat{\beta})^T X^T X (\beta - \hat{\beta})$$

The second term is a quadratic form in $(\beta - \hat{\beta})$. Since $X^T X$ is positive (semi-)definite, its level sets $\{\beta : \text{RSS}(\beta) = k\}$ are ellipsoids centered at $\hat{\beta}$ — in two dimensions, ellipses.

- The **shape** of the ellipse (how stretched or tilted) comes directly from the eigenstructure of $X^T X$, i.e. from how correlated the features are.
 - Uncorrelated, similarly-scaled features -> circular contours
 - Correlated features -> stretched, tilted ellipses
- As k increases you get bigger ellipses, all nested, all centered on $\hat{\beta}$. That's the "ellipses expanding outward from the OLS point" in the usual picture.

Where the constraint region comes from

The objective you actually fit — `Ridge(alpha...)` or `Lasso(alpha...)` — is the *penalized* (Lagrangian) form:

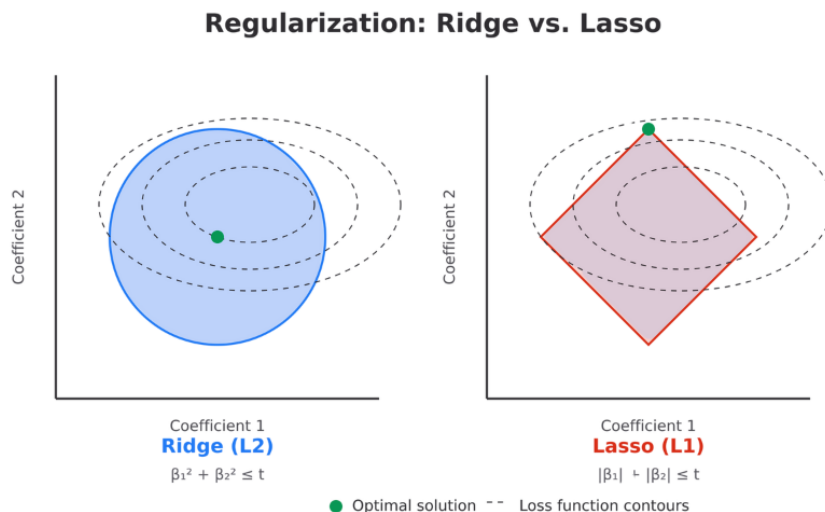
$$\min_{\beta} \text{RSS}(\beta) + \alpha \cdot P(\beta), \quad P(\beta) = \sum_j \beta_j^2 \text{ (ridge) or } \sum_j |\beta_j| \text{ (lasso)}$$

For a convex problem, this is exactly the Lagrangian of the *constrained* form:

$$\min_{\beta} \text{RSS}(\beta) \quad \text{subject to} \quad P(\beta) \leq c$$

with α playing the role of the Lagrange multiplier. KKT theory guarantees a one-to-one correspondence between α and c : solving the penalized problem for a given α lands you on the same β^* you'd get by solving the constrained problem for the matching c (larger $\alpha \Leftrightarrow$ smaller c , a tighter region). So "add a penalty term" and "shrink the feasible region and stay inside it" are the *same* optimization problem viewed two ways — the constraint region isn't a metaphor, it's the dual formulation of what scikit-learn is literally solving.

$P(\beta) \leq c$ traces out a circle for the L2 penalty ($\sum_j \beta_j^2 \leq c$) and a diamond for the L1 penalty ($\sum_j |\beta_j| \leq c$) — the shape of each norm's unit ball, scaled by c .



Why the touch point differs

The constrained optimum is the point in the feasible region with the smallest RSS — geometrically, the point where the smallest (innermost) RSS ellipse still touches the constraint boundary, since $\hat{\beta}$ itself sits outside the region whenever $\alpha > 0$.

- The **diamond** is a polytope with corners exactly on the coordinate axes (where one coordinate is 0). The **circle** is smooth everywhere.
- An expanding smooth ellipse touching a polytope will, for generic ellipse orientations, make first contact at a **vertex** rather than along a flat edge — touching an edge tangentially would require the ellipse's axis to align exactly with that edge, a measure-zero coincidence.
- Touching a smooth circle, by contrast, generically happens at an ordinary boundary point with no coordinate forced to zero.

The rigorous version of "generically touches a corner" goes through subgradients: $|x|$ isn't differentiable at 0, so its subdifferential there is the whole interval $[-1, 1]$ rather than a single slope. That gives the KKT stationarity condition a lot of room to be satisfied exactly at $\beta_j = 0$, which is why Lasso zeroes out a coefficient over a whole *range* of data, not just as a fluke. Ridge's smooth $2\beta_j$ gradient only vanishes exactly at $\beta_j = 0$ itself, which real data essentially never produces exactly.

That's the full answer to "why does Lasso produce exact zeros?" — not special logic in the algorithm, just the geometry (and subgradient calculus) of an L1 ball having corners where an L2 ball doesn't.

A practical corollary: scale your features first

Both penalties are defined directly on the raw coefficient values β_j , with no reference to the scale of the feature each coefficient belongs to. A feature that only ranges over 0-1 needs a large coefficient to have any effect on predictions; a feature ranging over 0-1000 needs a small one. Since the penalty punishes coefficient **size**, not predictive importance, an unscaled Lasso fit can zero out a genuinely useful small-scale feature simply because its coefficient had to be large to compensate for its small range — while a large-scale, less useful feature survives because its coefficient is already small. Standardizing features before fitting (e.g. with `StandardScaler`, ideally inside a `Pipeline`) puts every coefficient on the same footing, so the penalty compares like with like.

References

- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2nd ed.), Section 6.2.2. The source

of the circle/diamond figure used throughout this note. <https://www.statlearning.com/>

- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.), Section 3.4. The more technical treatment, including the Lagrangian/constrained equivalence. <https://hastie.su.domains/ElemStatLearn/>
- Tibshirani, R. (1996). Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society, Series B*, 58(1), 267-288. The original Lasso paper.